

TP2 – Pandas & NumPy avancés

M1 Data & IA – Université Catholique de Lille

Rendu attendu

- Un notebook Jupyter `.ipynb` exécutable de bout en bout.
- Travail en binôme conseillé.
- Pour chaque question, la réponse doit apparaître dans une cellule **Markdown** placée sous le code.
- Nom du fichier : `TP2_Pandas_Avancé_NOM_Prenom.ipynb`

Contexte

Votre responsable à la foncière lilloise est satisfait de votre analyse DVF de la séance 1. Il vous confie maintenant une mission plus ambitieuse : **enrichir le dataset avec les données démographiques INSEE des communes du Nord**, identifier des corrélations entre densité de population et niveau de prix, et produire des tableaux de synthèse pour la réunion de direction.

Mais d’abord, il a lu dans un article qu’un data engineer junior avait corrompu une base de production à cause d’un bug silencieux Pandas. Il vous demande de lui prouver que vous maîtrisez ce piège – et que vous savez écrire du code performant. Ce TP vous fait passer de « *ça fonctionne* » à « *ça fonctionne bien, vite et proprement* ».

Jeux de données

- **Source 1** : `dvf_clean.parquet` – votre fichier Parquet produit en TP1.
Si le fichier est absent : rechargez le CSV DVF Nord 2023 depuis data.gouv.fr et relancez votre pipeline TP1.
- **Source 2** : Populations légales communes
`geo.api.gouv.fr/departements/59/communes?fields=nom,code,population&format=json`
Colonnes : `code` (code INSEE), `nom` (nom commune), `population` (PMUN).

Objectifs pédagogiques

À l’issue du TP, vous devez être capables de :

- identifier et corriger le bug *Copy vs View* de Pandas ;
- quantifier expérimentalement le gain de la vectorisation sur vos données ;
- transformer un dataset entre les formats *wide* et *long* (Tidy Data) ;
- réaliser une jointure sur clé imparfaite avec diagnostic systématique ;
- optimiser les types d’un DataFrame (`category`, `Int64`, `datetime64`) ;
- exécuter une première requête avec Polars en mode *lazy*.

1. Mise en place

1.1. Structure du notebook

Créez dès le départ les titres suivants dans votre notebook :

```
# TP2 -- Pandas & NumPy avans

## 0. Imports et configuration
## 1. Architecture & Performance
## 2. Reshaping -- Tidy Data
## 3. Enrichissement par jointure INSEE
## 4. Types avans
## 5. Introduction Polars
## 6. Synthse (optionnelle)
```

1.2. Imports et configuration

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import time, os, warnings
import unicode # pip install unicode

warnings.filterwarnings("ignore")
pd.set_option("display.max_columns", 50)
pd.set_option("display.float_format", "{:,.0f}".format)
```

1.3. Chargement du dataset

Deux chemins selon votre situation

- Vous avez un `dvf_clean.parquet` issu du TP1 : utilisez le **Chemin A** (une seule ligne).
- Vous n’avez pas terminé le TP1 (ou votre Parquet est manquant) : utilisez le **Chemin B**, qui est un pipeline autonome complet. Exécutez-le du début à la fin avant de passer à la partie 1.

Chemin A – Vous avez le Parquet du TP1

```
df = pd.read_parquet("dvf_clean.parquet")
print(f"Dataset charge : {df.shape[0]:,} lignes x {df.shape[1]} colonnes")
df.head(3)
```

Chemin B – Pipeline autonome (TP1 non termine)

Copiez et exécutez l’intégralité du bloc ci-dessous. Il télécharge les données brutes, applique un nettoyage standard et produit le fichier `dvf_clean.parquet` nécessaire pour la suite du TP.

```
# =====
# BLOC DE SECOURS -- a executer si dvf_clean.parquet est absent
# =====

import numpy as np
import pandas as pd
import os

# ---- 1. EXTRACT -----
```

```

def extract(url: str) -> pd.DataFrame:
    print("Telechargement du dataset DVF Nord 2023...")
    df = pd.read_csv(url, sep=";", low_memory=False)
    print(f" -> {df.shape[0]:,} lignes, {df.shape[1]} colonnes chargees")
    return df

# ---- 2. TRANSFORM -----
def transform(df: pd.DataFrame) -> pd.DataFrame:
    print(f"Transformation : {len(df):,} lignes en entree")

    # Correction des types
    df["date_mutation"] = pd.to_datetime(df["date_mutation"], errors="coerce")
    df["code_commune"] = df["code_commune"].astype(str)

    # Suppression des doublons
    n_avant = len(df)
    df = df.drop_duplicates()
    print(f" Doublons supprimes : {n_avant - len(df)}")

    # Suppression des lignes sans valeur fonciere
    df = df.dropna(subset=["valeur_fonciere"])

    # Calcul du prix au m2 (appartements uniquement pour le nettoyage)
    df["prix_m2"] = df["valeur_fonciere"] / df["surface_reelle_bati"]

    # Filtre des valeurs aberrantes
    df_clean = df[
        (df["valeur_fonciere"] > 0) &
        (df["valeur_fonciere"] < 5_000_000) &
        (df["prix_m2"].isna() | df["prix_m2"].between(100, 30_000))
    ].copy()

    print(f" -> {len(df_clean):,} lignes en sortie apres nettoyage")
    return df_clean

# ---- 3. LOAD -----
def load(df: pd.DataFrame, dest: str = "dvv_clean.parquet") -> None:
    df.to_parquet(dest, compression="snappy", index=False)
    taille = os.path.getsize(dest) / 1024
    print(f"Fichier sauvegarde : {dest} ({taille:,.0f} Ko)")

# ---- EXECUTION DU PIPELINE -----
URL = ("https://files.data.gouv.fr/geo-dvf/latest/csv/"
      "2023/departements/59.csv.gz")

df_raw = extract(URL)
df_clean = transform(df_raw)
load(df_clean, "dvv_clean.parquet")

# Rechargement pour la suite du TP
df = pd.read_parquet("dvv_clean.parquet")
print(f"\nDataset pret : {df.shape[0]:,} lignes x {df.shape[1]} colonnes")
# =====

```

Question 1. Vérifiez que les types sont corrects (`df.dtypes`), notamment pour `date_mutation` et

`valeur_fonciere`. Appliquez les corrections nécessaires avant d'aller plus loin.

Indication : si vous avez utilisé le Chemin B, ces corrections sont déjà appliquées par le pipeline. Contentez-vous de vérifier avec `df.dtypes` et de documenter ce que vous observez en Markdown.

2. Architecture & Performance

2.1. Benchmark – Boucle vs `apply()` vs vectorisé

Une opération vectorisée est bien plus rapide qu'une boucle Python. Mais `apply()` est-il vraiment une bonne alternative à une boucle ? Vérifiez-le expérimentalement – le résultat peut vous surprendre.

Point de départ

Le benchmark porte sur le calcul `valeur_fonciere * 1.20` sur 50 000 lignes. Répétez chaque mesure 5 fois et prenez la médiane. Ajoutez également une mesure du surcoût interne de `apply()` seul (fonction identité).

```
def bench(func, n=5):
    times = []
    for _ in range(n):
        t0 = time.perf_counter()
        func()
        times.append(time.perf_counter() - t0)
    return round(sorted(times)[n // 2] * 1000, 2) # mediane en ms

sample = appartements.head(50_000).copy()

t_boucle = bench(lambda: [v * 1.20 for v in sample["valeur_fonciere"]])
t_apply = bench(lambda: sample["valeur_fonciere"].apply(lambda x: x * 1.20))
t_vec = bench(lambda: sample["valeur_fonciere"] * 1.20)
t_overhead = bench(lambda: sample["valeur_fonciere"].apply(lambda x: x)) # identite

print(f"Boucle Python : {t_boucle:>8.2f} ms")
print(f"apply() : {t_apply:>8.2f} ms")
print(f"Vectorisé : {t_vec:>8.2f} ms")
print(f"Surcoût apply() (identite): {t_overhead:>8.2f} ms")
print()
print(f"apply() plus lent que boucle ? {t_apply > t_boucle}")
```

Question 4. Complétez le tableau suivant avec vos valeurs réelles. `apply()` est-il plus rapide ou plus lent que la boucle Python ? Expliquez ce résultat en tenant compte du surcoût mesuré avec la fonction identité.

Méthode	Temps (ms)	Ratio vs vectorisé
Boucle Python	???	???×
<code>apply()</code>	???	???×
Vectorisé	???	1×
Surcoût <code>apply()</code> seul	???	–

Indication : en Pandas 2.0+, `apply()` ajoute un surcoût interne fixe (gestion du Copy-on-Write, vérification des types, dispatch) qui s'applique même lorsque la fonction ne fait rien. Pour une opération scalaire simple, ce surcoût dépasse le coût de la boucle Python elle-même. `apply()` n'est

donc pas « une boucle optimisée » – c’est une boucle avec overhead supplémentaire, utile uniquement pour des logiques complexes multi-colonnes impossibles à vectoriser.

Question 5. Calculez une catégorie de prix pour chaque appartement : "entrée de gamme" si `prix_m2 < 2000` , "milieu de gamme" entre 2000 et 4000 , "haut de gamme" au-delà. Implémentez avec `np.select()` et mesurez le temps. Pourquoi cette approche est-elle préférable à `apply()` ?

2.2. Optimisation mémoire – dtype category

Question 6. Identifiez les colonnes candidates au type `category` (moins de 5 % de valeurs uniques). Convertissez-les et mesurez l’économie mémoire totale en MB et en pourcentage.

```
mem_avant = df.memory_usage(deep=True).sum() / 1024**2
print(f"Memoire avant : {mem_avant:.1f} MB")

for col in df.select_dtypes("object").columns:
    n_unique = df[col].nunique()
    pct = n_unique / len(df) * 100
    print(f"{col:<35} {n_unique:>6} valeurs uniques ({pct:.1f} %)")
```

Indication : complétez la liste `cols_cat` selon votre inspection, puis mesurez après conversion.

```
cols_cat = ["type_local", "nature_mutation"] # a completer
df[cols_cat] = df[cols_cat].apply(lambda c: c.astype("category"))

mem_apres = df.memory_usage(deep=True).sum() / 1024**2
print(f"Memoire apres : {mem_apres:.1f} MB")
print(f"Economie : {mem_avant - mem_apres:.1f} MB "
      f"({(1 - mem_apres / mem_avant) * 100:.0f} %)")
```

3. Reshaping – Tidy Data

3.1. Rapport trimestriel avec `pivot_table()`

Votre responsable veut un tableau croisé du prix médian par type de bien et par trimestre, prêt à être copié dans une présentation.

```
df["annee"] = df["date_mutation"].dt.year
df["trimestre"] = df["date_mutation"].dt.quarter
df["mois"] = df["date_mutation"].dt.month

rapport = df.pivot_table(
    values = "valeur_fonciere",
    index = "type_local",
    columns = "trimestre",
    aggfunc = "median",
    fill_value = 0,
)
rapport.columns = [f"T{q}" for q in rapport.columns]
rapport["Mediane annuelle"] = rapport.mean(axis=1).round(0)
rapport
```

Question 8. Tracez un graphique en barres groupées (`kind="bar"`) du tableau `rapport`. Ajoutez un titre, des labels d’axes et une légende lisibles. Sauvegardez l’image avec `plt.savefig("rapport_trimestriel.png", dpi=150, bbox_inches="tight")`.

3.2. Du format Wide au format Long avec melt()

```

rapport_reset = rapport.reset_index() # type_local devient une colonne

rapport_long = rapport_reset.melt(
    id_vars = "type_local",
    value_vars = ["T1", "T2", "T3", "T4"],
    var_name = "trimestre",
    value_name = "prix_median",
)

print(f"Format wide : {rapport_reset.shape}")
print(f"Format long : {rapport_long.shape}")
rapport_long.head(8)

```

Question 9. Combien de lignes et de colonnes a le DataFrame en format long par rapport au format wide? Pourquoi dit-on que le format long est *tidy* au sens de Wickham? Quel avantage offre-t-il pour les visualisations Seaborn?

```

plt.figure(figsize=(12, 5))
sns.barplot(
    data = rapport_long[rapport_long["prix_median"] > 0],
    x = "trimestre",
    y = "prix_median",
    hue = "type_local",
)
plt.title("Prix median par type de bien et trimestre -- Nord 2023")
plt.ylabel("Prix median (EUR)")
plt.tight_layout()
plt.show()

```

Question 10. Identifiez le trimestre où le prix médian des appartements est le plus élevé. Formulez une hypothèse sur cette saisonnalité du marché immobilier.

3.3. groupby() avancé avec transform()

```

apparts["med_commune"] = (
    apparts.groupby("nom_commune")["prix_m2"]
    .transform("median")
)
apparts["above_median"] = apparts["prix_m2"] > apparts["med_commune"]

summary = (
    apparts.groupby("nom_commune")
    .agg(
        n_ventes = ("valeur_fonciere", "count"),
        prix_med = ("prix_m2", "median"),
        pct_premium = ("above_median", "mean"),
    )
    .sort_values("n_ventes", ascending=False)
    .head(15)
)
summary

```

Question 11. Quelle différence fondamentale y a-t-il entre `agg()` et `transform()`? Dans quel cas `transform()` est-il irremplaçable? Répondez en Markdown avec un exemple concret.

Question 12. Tracez un graphique croisant le nombre de ventes (axe x) et le prix médian au m² (axe y) pour les communes du tableau `summary`. Faites varier la taille des points selon `pct_premium`. Les communes les plus actives sont-elles aussi les plus chères ?

4. Enrichissement par jointure INSEE

4.1. Chargement des données de population

Point de départ

Le fichier INSEE contient les populations légales 2020 par commune. Si l'URL ne répond pas, contactez l'enseignant pour le fichier local.

```
import requests

url = "https://geo.api.gouv.fr/departements/59/communes?fields=nom,code,population&format=json"
data = requests.get(url, timeout=30).json()

df_pop_59 = pd.DataFrame(data)[["code", "nom", "population"]].rename(columns={
    "code": "CODGEO",
    "nom" : "LIBGEO",
    "population" : "PMUN",
})
df_pop_59["PMUN"] = pd.to_numeric(df_pop_59["PMUN"], errors="coerce")

print(f"Communes Nord : {len(df_pop_59)}")
print(f"Population totale Nord : {df_pop_59['PMUN'].sum():,.0f}")
df_pop_59.head(5)
```

Question 13. Combien de communes du département 59 sont présentes ? Quelle est la commune la plus peuplée ? La moins peuplée ?

4.2. Normaliser les clés

Le dataset DVF et le fichier INSEE ne formatent pas les noms de communes de la même façon. Vous allez les harmoniser avant la jointure.

```
import unicodecode

def normalize_commune(s: str) -> str:
    """Normalise un nom de commune pour la jointure."""
    if pd.isna(s):
        return ""
    s = unicodecode.unicodecode(str(s)) # supprime les accents
    s = s.upper().strip() # majuscules, espaces bord
    s = s.replace(" ", "-") # espaces -> tirets
    return s

apparts["cle_commune"] = apparts["nom_commune"].apply(normalize_commune)
df_pop_59["cle_commune"] = df_pop_59["LIBGEO"].apply(normalize_commune)

print(apparts["cle_commune"].value_counts().head(8))
print(df_pop_59["cle_commune"].value_counts().head(8))
```

Question 14. Les formats correspondent-ils maintenant ? Identifiez un cas particulier (arrondissements, communes fusionnées, abréviations *ST* vs *SAINT*) qui risque de faire échouer la jointure.

4.3. La jointure et son diagnostic obligatoire

```
n_avant = len(apparts)

apparts_enrichis = pd.merge(
    apparts,
    df_pop_59[["cle_commune", "PMUN", "CODGEO"]],
    on = "cle_commune",
    how = "left",
)

n_apres = len(apparts_enrichis)

taux_match = apparts_enrichis["PMUN"].notna().mean() * 100
n_nan = apparts_enrichis["PMUN"].isna().sum()

print(f"Lignes avant jointure : {n_avant:,}")
print(f"Lignes apres jointure : {n_apres:,}")
print(f"Taux de correspondance : {taux_match:.1f} %")
print(f"Appartements sans pop : {n_nan:,}")

communes_sans_match = (
    apparts_enrichis[apparts_enrichis["PMUN"].isna()]
    ["cle_commune"].value_counts().head(10)
)
print(communes_sans_match)
```

Question 15. Quel est votre taux de correspondance ? Si `n_apres > n_avant`, expliquez pourquoi et comment corriger le problème. Listez les 3 communes les plus actives sans correspondance.

Indication : si `n_apres > n_avant`, vérifiez les doublons dans `df_pop_59` avec `df_pop_59["cle_commune"].duplicated()`

Question 16. Choisissez la stratégie la mieux adaptée pour les appartements sans correspondance (NaN dans PMUN) : imputation, suppression ou catégorie « Non apparié ». Justifiez votre choix en Markdown, puis implémentez-le.

4.4. Analyse enrichie – Prix et démographie

```
bins = [0, 5_000, 20_000, 100_000, float("inf")]
labels = ["Rural (<5k)", "Petite ville (5-20k)",
          "Ville (20-100k)", "Grande ville (>100k)"]

apparts_enrichis["categorie_pop"] = pd.cut(
    apparts_enrichis["PMUN"],
    bins = bins,
    labels = labels,
    right = False,
)

stats_pop = (
    apparts_enrichis.groupby("categorie_pop", observed=True)
    .agg(
```



```

    n_ventes = ("valeur_fonciere", "count"),
    prix_med = ("prix_m2", "median"),
    prix_moy = ("prix_m2", "mean"),
    surf_med = ("surface_reelle_bati", "median"),
)
    .round(0)
)
stats_pop

```

Question 17. Décrivez la relation entre taille de la commune et prix au m². Est-elle linéaire ? Tracez un boxplot `prix_m2` par `categorie_pop` avec Seaborn.

Question 18. Rédigez en Markdown un paragraphe d'environ 100 mots, à l'attention de votre responsable non technicien, synthétisant les résultats de cette analyse démographique. Citez des chiffres concrets.

5. Types avancés

5.1. datetime avancé – Extractions et arithmétique temporelle

```

print(df["date_mutation"].dtype)

df["jour_semaine"] = df["date_mutation"].dt.day_of_week # 0 = Lundi
df["semaine_iso"] = df["date_mutation"].dt.isocalendar().week

```

Question 19. Tracez la distribution des transactions par jour de la semaine (barplot). Le marché immobilier du Nord présente-t-il un rythme hebdomadaire ? Quelle explication professionnelle proposez-vous ?

Question 20. Calculez le prix médian des appartements pour chaque mois, puis identifiez les 3 mois les plus actifs (volume) et les 3 mois les plus chers (prix médian). Ces deux classements coïncident-ils ? Que cela implique-t-il pour un acheteur ?

5.2. Nullable integers – Corriger les colonnes identifiants

En rechargeant un CSV, certaines colonnes d'identifiants numériques sont converties en `float64` à cause de valeurs manquantes. Le type nullable `Int64` résout ce problème.

```

for col in df.select_dtypes("float64").columns:
    if any(k in col.lower() for k in ["id", "numero", "code"]):
        n_nan = df[col].isna().sum()
        print(f"{col:<35} NaN: {n_nan:>6} -> candidat Int64")

```

Question 21. Convertissez au moins deux colonnes identifiants en `Int64` (nullable). Vérifiez que les NaN sont préservés sous forme de `pd.NA`. Quelle différence concrète cela fait-il pour un code qui teste l'égalité `id == 12345` ?

Indication : `df[col] = df[col].astype("Int64")` (majuscule). `np.nan` est un float ; `pd.NA` est le vrai « not available » agnostique du type.

6. Introduction à Polars

Installation

```
pip install polars
```

Polars est indépendant de Pandas. Les deux peuvent coexister dans le même notebook.

6.1. Chargement et exploration

```
import polars as pl

df_pl = pl.read_parquet("dvf_clean.parquet")

print(df_pl.shape)
print(df_pl.dtypes)
df_pl.head(3)
```

Question 22. Comparez les types Polars avec les types Pandas de votre DataFrame `df`. Quelles différences observez-vous, notamment pour les colonnes catégorielles et les dates ?

6.2. Syntaxe Polars vs Pandas

```
# Filtrage + nouvelle colonne + filtre + groupby en mode Lazy
stats_pl = (
    df_pl.lazy()
    .filter(pl.col("type_local") == "Appartement")
    .with_columns(
        (pl.col("valeur_fonciere") / pl.col("surface_reelle_bati"))
        .alias("prix_m2")
    )
    .filter(pl.col("prix_m2").is_between(500, 15_000))
    .group_by("nom_commune")
    .agg([
        pl.len().alias("n_ventes"),
        pl.col("prix_m2").median().alias("prix_med"),
        pl.col("prix_m2").mean().alias("prix_moy"),
    ])
    .sort("n_ventes", descending=True)
    .collect()
)
stats_pl.head(10)
```

Question 23. Qu'est-ce que `.lazy()` change dans le comportement de Polars ? Expliquez en Markdown le concept de *predicate pushdown* en une phrase concrète appliquée à ce code.

Indication : le filtre `is_between(500, 15 000)` est « remonté » avant le `with_columns` – Polars ne calcule `prix_m2` que pour les lignes qui passeront le filtre.

6.3. Benchmark – Pandas vs Polars

```
def bench(func, n=3):
    times = [None] * n
    for i in range(n):
```

```

    t0 = time.perf_counter()
    func()
    times[i] = time.perf_counter() - t0
    return round(sorted(times)[1] * 1000, 1) # mediane en ms

def groupby_pandas():
    return (
        apparts.groupby("nom_commune")
        .agg(n_ventes=("valeur_fonciere", "count"),
             prix_med=("prix_m2", "median"),
             prix_moy=("prix_m2", "mean"))
    )

def groupby_polars():
    return (
        pl.from_pandas(appartss).lazy()
        .group_by("nom_commune")
        .agg([pl.len().alias("n_ventes"),
              pl.col("prix_m2").median().alias("prix_med"),
              pl.col("prix_m2").mean().alias("prix_moy")])
        .collect()
    )

t_pd = bench(groupby_pandas)
t_pl = bench(groupby_polars)

print(f"Pandas : {t_pd:>8.1f} ms")
print(f"Polars : {t_pl:>8.1f} ms")
print(f"Polars est x{t_pd / t_pl:.1f} plus rapide")

```

Question 24. Complétez le tableau suivant avec vos mesures réelles. Le gain observé correspond-il à l'ordre de grandeur annoncé dans le cours (5 à 10×) ?

Bibliothèque	Temps (ms)	Ratio
Pandas	???	1×
Polars Lazy	???	???×

Question 25. Dans quels cas continueriez-vous à utiliser Pandas plutôt que Polars, même si Polars est plus rapide ? Citez au moins deux situations concrètes tirées de ce TP ou du cours.

7. Synthèse (optionnelle)

Cette partie est optionnelle

Elle n'est pas comptabilisée dans la note de base. Elle est fortement conseillée si vous avez terminé les parties 1 à 5 – elle démontre votre capacité à lier pratique et théorie, et distingue les meilleurs rendus.

7.1. Rapport technique

Question 26. Rédigez un rapport d'environ 150 mots à l'attention de votre responsable. Organisez-le ainsi :

1. **Performance** – ce que le benchmark vous a appris, avec le ratio observé entre boucle, Pandas et Polars.
2. **Enrichissement** – taux de correspondance INSEE et traitement des cas non appariés.
3. **Recommandation** – dans quel cas basculeriez-vous ce pipeline vers Polars en production ? Justifiez en deux phrases.

Le texte doit être compréhensible par une personne qui ne code pas.

Conseils de présentation

- Structurez le notebook avec des titres clairs correspondant aux sections de ce sujet.
- Commentez vos choix lorsqu'ils ont un impact sur l'analyse (choix de seuils, stratégie de jointure, etc.).
- Un bon notebook explique autant qu'il calcule : privilégiez des cellules Markdown rédigées.
- Soignez la lisibilité des graphiques : titre, axes, unités, légende si nécessaire.
- Vérifiez que le notebook s'exécute du début à la fin sans erreur avant de rendre.